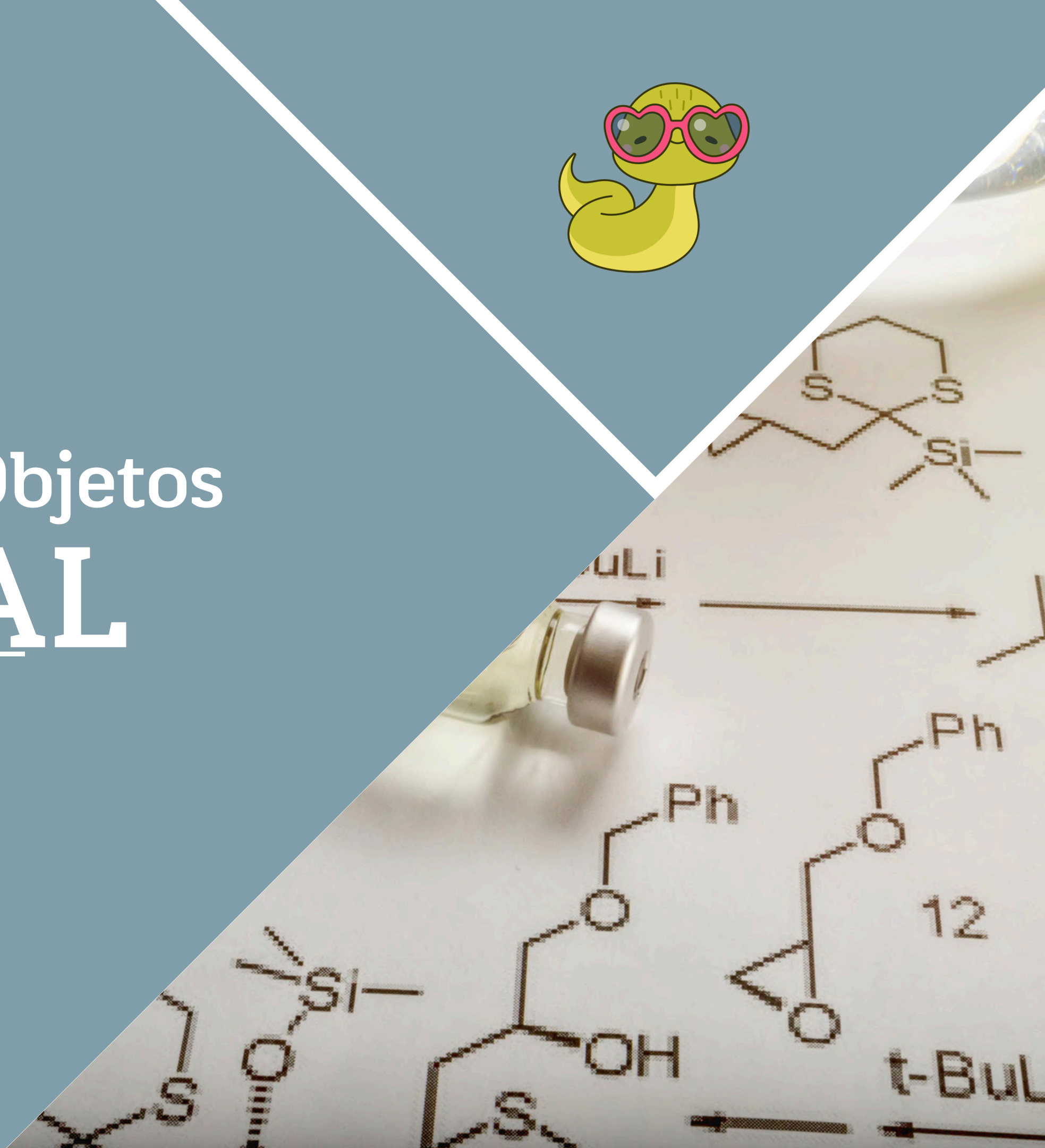




Programação Orientada a Objetos

PROJETO FINAL



PROJETO FINAL

- Modelagem de domínio: transformar entidades do mundo real em classes, atributos e métodos.
- Encapsulamento: definir atributos privados, usar getters/setters ou @property para proteger o estado.
- Herança: criar hierarquias quando fizer sentido (superclasses + subclasses), evitando duplicação.
- Polimorfismo: sobrescrever métodos nas subclasses e usar duck-typing para código genérico.
- Separação de responsabilidades: organizar o projeto em modelo (classes), controle (lógica de negócio) e interface (menu/entrada e saída).
- Validação e regras de negócio: aplicar verificações dentro de setters/métodos (ex.: não emprestar item indisponível).
- Interação via menu: construir uma interface de console simples que chama a camada de controle, sem manipular diretamente os dados.
- Boas práticas: clareza, coesão das classes, comentários explicativos e testes manuais via menu.

1. Sistema de Gerenciamento de Biblioteca

1. Simular o funcionamento básico de uma biblioteca: cadastrar mídias (livros, revistas, filmes), controlar disponibilidade e empréstimos, manter histórico e permitir buscas. Aplicação em camadas (modelo, controle, interface) e menu interativo.

- Deve conter (mínimo obrigatório)
 - ✓ Classe pai Midia com atributos comuns (título, autor, ano, id)
 - ✓ Subclasses Livro, Revista, Filme com atributos específicos (ex.: ISBN, edição, duração)
 - ✓ Encapsulamento para atributo __disponivel (privado)
 - ✓ Métodos emprestar(contato) e devolver() que atualizam disponibilidade e histórico básico
 - ✓ Polimorfismo: método obter_tipo() que retorna o tipo da mídia (implementado diferente nas subclasses)
 - ✓ Lista de mídias cadastradas e lista (ou registro) de empréstimos ativos
 - ✓ Menu interativo para: cadastrar, listar, buscar (por título/autor/tipo), emprestar, devolver, remover

2.Sistema de Controle de Estoque para Loja

1. Criar um sistema para cadastrar produtos, controlar estoque (entradas/saídas), buscar produtos e listar produtos em falta. Organização em modelo/controle/interface e menu interativo.

- Deve conter (mínimo obrigatório)
 - ✓ Classe Produto com atributos (nome, código, preço, __quantidade) — quantidade encapsulada
 - ✓ Getters/setters ou @property para acessar/alterar quantidade com validação (não negativa)
 - ✓ Métodos adicionar_estoque(qtd) e remover_estoque(qtd) que atualizam e validam estoque
 - ✓ Lista de produtos cadastrados e consulta por nome
 - ✓ Menu interativo para: cadastrar produto, atualizar estoque (entrada/saída), listar todos, listar produtos com estoque baixo, remover produto

3. Sistema de Gerenciamento de Alunos, Turmas e Notas

1. Desenvolver um sistema educacional simples que gerencie alunos, turmas e notas. Permitir cadastro, associação aluno $\leftrightarrow$ turma, lançamento de notas, cálculo de médias e relatórios de aprovação.

- Deve conter (mínimo obrigatório)
 - ✓ Classe Pessoa ou Usuario (opcional) e classe Aluno com atributos (nome, matrícula, __notas)
 - ✓ Classe Turma com atributos (nome, código, lista de alunos)
 - ✓ Encapsulamento das notas (__notas) e uso de getters/setters para lançar notas com validação (0–100)
 - ✓ Métodos para adicionar/remover aluno em turma, lançar nota, calcular média por aluno e por turma
 - ✓ Menu interativo para: cadastrar aluno, criar turma, matricular, lançar nota, listar aprovados/reprovados, buscar aluno

4. Sistema de Gerenciamento de Hotel

Criar um sistema para gerenciar quartos, reservas e hóspedes. Funcionalidades mínimas: cadastrar quarto, reservar, calcular valor de estadia e registrar pagamentos. Organização em camadas e menu interativo.

- Deve conter (mínimo obrigatório)
 - ✓ Classe Quarto (atributos: número, tipo, diária, __ocupado) — encapsular estado de ocupação
 - ✓ Classe Reserva relacionando Hóspede + Quarto + datas (check-in/check-out) e cálculo do valor total
 - ✓ Controller com regras: reservar somente se quarto disponível, liberar quarto na devolução
 - ✓ Encapsulamento e validação nas reservas (datas coerentes)
 - ✓ Menu interativo: cadastrar quarto, listar quartos, criar reserva, cancelar reserva, consultar reservas ativas

5) Sistema de Gerenciamento de Pets e Consultas em Clínica Veterinária

1. Desenvolver um sistema para cadastrar pets e donos, agendar consultas, registrar atendimento e manter histórico. Aplicação estruturada em modelo/controle/interface, com menu interativo.

- Deve conter (mínimo obrigatório)
 - ✓ Classe Pessoa (dono) e Pet (atributos: nome, espécie, __idade, proprietário)
 - ✓ Herança para tipos de Pet (ex.: Cachorro, Gato, Passaro) com comportamento/atributos específicos se necessário
 - ✓ Encapsulamento de informações sensíveis (ex.: histórico médico privado) com getters/setters ou @property para leitura controlada
 - ✓ Classe Consulta com atributos (pet, data, motivo, realizado) e métodos para agendar e finalizar consulta
 - ✓ Menu interativo para: cadastrar dono, cadastrar pet, agendar consulta, listar próximas consultas, registrar atendimento